

ACME 30 – Assignment no. 3: Security Policy Definition and Implementation

Alberto Pulcini 2046542
Nicolò Piovan 2219663
Giovanni Colasuonno 2046000
Davide Nardoni 2060903

June 17, 2026

1 Introduction

This report describes the work carried out by our group (ACME 30) for the third assignment: defining and enforcing a *security policy* on the ACME network. The activity was organised in the following phases:

1. **Brainstorming**, where we classified the network zones by trust level and chose a *default-deny, least-privilege* approach following NIST SP 800-41 [1].
2. **Service configuration**, where every service required by the assignment was brought up with a minimal but realistic setup (HTTPS web service, DNS, centralized logging, forward/reverse proxy, log collector, vulnerability scanner, vending machine).
3. **Policy definition**, where we wrote the inter-zone matrix and the list of the flows that are actually needed, *before* touching any rule.
4. **Policy implementation**, where the matrix was translated into readable, alias-based rules on the Main and Internal firewalls, closed by an explicit default-deny on every interface.
5. **Tests**, to verify that each allowed flow works and each forbidden one is blocked and logged.
6. **Final remarks**.

2 Brainstorming

We started by labelling each zone of the ACME network with a trust level, which directly drives how permissive its rules can be (Table 1). The full topology is recalled in Figure 8.

Zone	Firewall	Trust
Internet / WAN	Main	Untrusted
VPN road warriors	Main	Per-role (A2)
DMZ	Main	Exposed / semi
External services	Main	Low
Internal servers	Internal	High
Clients	Internal	Medium

Table 1: Zones and trust levels.

The guiding principles were: **default-deny** (anything not explicitly required is blocked and logged); **public services only through a single entry point**; **proxy-enforced** Internet access for internal hosts; **centralized logging**; **restricted administration**; and a **scanner** (Greenbone) allowed to reach every zone but never reachable itself.

Main design choice: reverse proxy. The proxy host in the DMZ is used as the *only* public entry point for the web services: the Internet talks to the reverse proxy, never to the real backends. This concentrates TLS termination and access control on one host, hides the backends, and keeps the firewall rules simple. The forward-proxy role (internal hosts → Internet) is served by the same host on a different port.

3 Configuration of the services

All seven services were configured (Table 2). Certificates are issued by the internal CA created in Assignment 2.

Service	Host	Notes
Web	.6.2	Apache, HTTPS + 80→443
DNS	.1.2	dnsmasq, A + AAAA
Syslog	.1.3	rsyslog UDP/514
Proxy	.6.3	Squid fwd + nginx rev
Graylog	.1.10	syslog 5140, GELF 12201
Greenbone	.1.4	GVM, HTTPS 9392
Vending	.4.10	HTTP, published via proxy

Table 2: ACME services. IPs abbreviated from 100.100.X.Y.

Web service (HTTPS only). Apache terminates TLS on 443 with a server certificate for `webserver.acme-30.test` (SANs covering the name, IPv4 and IPv6), while a port-80 vhost issues a permanent redirect to HTTPS:

```
$ curl -I http://localhost/  
HTTP/1.1 301 Moved Permanently  
Location: https://webserver.acme-30.test/  
$ curl -kI https://localhost/  
HTTP/1.1 200 OK  
Strict-Transport-Security: max-age=31536000
```

Forward proxy (clients → Internet). Squid on 3128 is the only authorised path for internal hosts to browse the web: the firewall blocks any direct HTTP/HTTPS egress, so a host that wants to reach the Internet *must* go through the proxy. Squid itself accepts requests only from the ACME subnets (source ACL), as a second line of defence on top of the firewall:

```
# /etc/squid/conf.d/acme.conf
acl acme_v4 src 100.100.1.0/24 100.100.2.0/24
acl acme_v4 src 100.100.4.0/24 100.100.6.0/24
acl acme_v6 src 2001:470:b5b8:1e81::/64 ...
http_access allow acme_v4
http_access allow acme_v6 # else: deny all
dns_v4_first on          # prefer IPv4 egress
```

This ACL is intentionally coarse (all ACME subnets): the *firewall* is the authoritative layer that decides which zones may reach the forward proxy at all (per Table 4, only Clients and Servers), while the ACL is a redundant host-level check. A client (or a tool such as apt) is pointed at the proxy explicitly; nothing else can leave the network: nothing else can leave the network:

```
export http_proxy=http://100.100.6.3:3128
export https_proxy=http://100.100.6.3:3128
# apt: /etc/apt/apt.conf.d/01proxy
Acquire::http::Proxy "http://100.100.6.3:3128";
```

For HTTP the proxy forwards the request and tags it with a Via header; for HTTPS it opens a CONNECT tunnel, so the payload stays encrypted end-to-end (the proxy sees only the destination host, not the content).

Reverse proxy (Internet → services). nginx on 80/443 is the single public entry point: it terminates TLS and forwards each request to the right backend, choosing the certificate by SNI (the hostname the client requests in the TLS handshake). Two server blocks, two certificates, two backends:

```
server { # site 1
    listen 443 ssl;
    server_name webserver.acme-30.test;
    ssl_certificate webserver.crt; # internal CA
    location / { proxy_pass https://100.100.6.2; }
}
server { # site 2
    listen 443 ssl;
    server_name fantasticcoffee.acme-30.test;
    ssl_certificate fantasticcoffee.crt;
    location / { proxy_pass http://100.100.4.10; }
}
# a port-80 server returns 301 -> https
```

The vending machine only speaks HTTP, so the reverse proxy is what gives it an HTTPS front-end without modifying the device. To keep DNS consistent with the policy, the internal DNS names of both web services resolve to the reverse proxy (.6.3) instead of to the backends; the backends (.6.2, .4.10) are reached only by nginx, by IPv4 literal. So a client opening https://webserver.acme-30.test is sent by DNS to the proxy, the firewall allows client → proxy (but not client → backend), nginx decrypts, selects the

backend and re-issues the request: the real servers are never exposed directly. For the web service this means TLS is terminated *twice*: nginx decrypts the client connection and re-encrypts toward the Apache backend, which keeps its own certificate (the proxy-to-backend hop stays inside the trusted DMZ); the vending machine, having no TLS, is reached in plain HTTP on the same segment.

Centralized logging. The log server stores one file per source host (/var/log/remote/<host>/) and Graylog exposes a Syslog (5140) and a GELF (12201) input. A test message from a host is correctly received and indexed (Figure 1). As hardening, an encrypted channel (*syslog over TLS*, RFC 5425 [3]) was added on TCP/6514: the web server forwards its logs to the log server over TLS, verifying the server certificate against the internal CA. A capture on 6514 shows only encrypted application data, unlike the plaintext UDP/514 baseline.

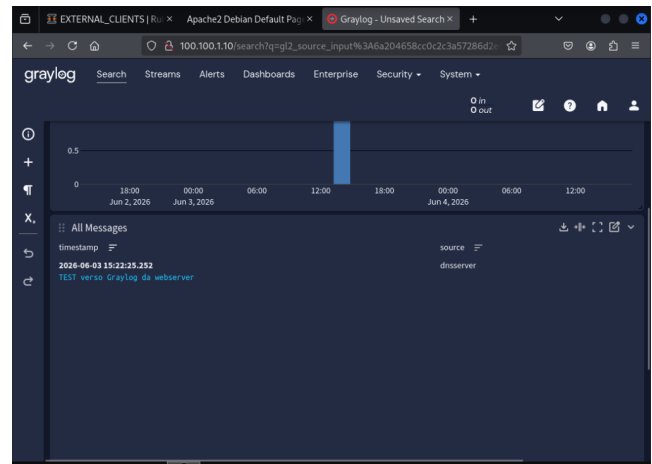


Figure 1. Graylog receiving a remote log message (centralized logging).

4 Definition of the security policy

The policy was written declaratively first. Table 3 is the inter-zone matrix (*R* = allowed but restricted to the flows below, *D* = deny); Table 4 lists the flows that the firewalls actually permit.

src \ dst	INET	DMZ	EXT	SRV	CLI
INET	—	R	D	D	D
DMZ	R	—	R	R	D
EXT	D	R	—	R	D
SRV	R	R*	R*	—	R*
CLI	D†	R	D	R	—

Table 3: Zone matrix. R* from SRV = Greenbone scan / proxy / DNS only; † CLI→INET only ICMP (no direct HTTP/HTTPS).

Source	Destination	Port
Internet	reverse proxy	80/443
External clients	reverse proxy	80/443
Clients	forward proxy	3128
Clients	reverse proxy	80/443
Clients/DMZ/Ext	DNS server	53
Clients/DMZ/Ext	log server	514, 6514
Clients/DMZ/Ext	Graylog	5140, 12201
Proxy	Internet	80/443
DNS server	Internet	53
Servers	forward proxy	3128
Greenbone	all ACME zones	any
admin (Kali)	firewalls GUI / Greenbone / Graylog	mgmt

Table 4: Allowed *application* flows. In addition, each interface permits ICMP/ICMPv6 (diagnostics, NDP, PMTUD) and the firewalls forward their own logs to Graylog (100.100.254.1→graylog:5140); everything else is dropped by default-deny.

Three asymmetries are deliberate. Clients have *no* direct Internet HTTP/HTTPS (only via the proxy); ICMP toward the Internet is left open for diagnostics, so CLI→INET is a deny *except* ping. Greenbone *initiates* the scans and never accepts connections, with the single exception of the administrator reaching its web UI. Finally, the administrative restriction is *host-level*, not zone-level: the matrix shows CLI→SRV = R for the whole client network, but the rules pin the management flows (firewall GUIs, Greenbone, Graylog UI) to the host_admin alias (Kali) only – the zone matrix alone cannot express this, the alias-based rules do.

5 Policy implementation

The matrix was implemented on both OPNsense firewalls. To keep the rules readable we used **aliases** (host_webserver, host_proxyserver, net_dmz, port_web, ...) instead of raw addresses. On each interface the old permissive any → any rule was disabled and replaced by the explicit pass rules of Table 4, closed by a final block log any → any.

Dual-stack. Because the network is dual-stack since Assignment 1, the service rules use inet46 (the OPNsense label for “IPv4+IPv6”) and the aliases carry both the IPv4 address and the IPv6 one; an explicit **ICMPv6** pass rule per interface allows the essential types (NDP, PMTUD), as recommended by RFC 4890 [2]. Without it, IPv6 would break even though every service answers on IPv6.

Default-deny on every interface. Each interface (WAN, DMZ, External, Internal, Clients, Servers) ends with a logging block rule. This both enforces the policy and produces the evidence used in the tests.

Firewalls as log sources. Both firewalls forward their packet-filter logs to Graylog (Syslog UDP/5140), so the pass/block decisions are centralized. Since the Main firewall

must cross the Internal one to reach Graylog, a dedicated pass rule (100.100.254.1 → graylog:5140) was added on the Internal firewall. Graylog receives filterlog events from both firewalls (Figure 2).

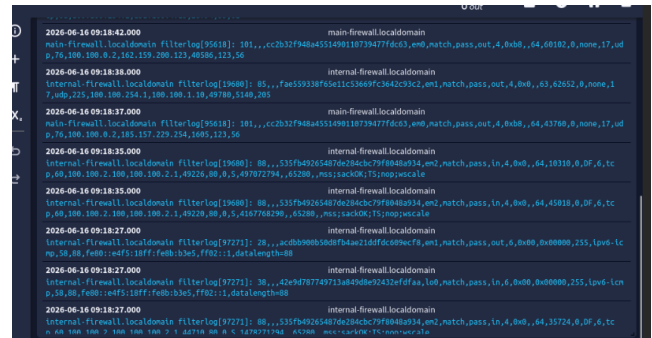


Figure 2. Filter logs of both firewalls collected in Graylog.

6 Test of the policy enforcement

For every flow we checked that allowed traffic passes and forbidden traffic is dropped and logged. The lab restricts Internet egress (only ping, plus an HTTP/HTTPS domain whitelist), so reachability tests use a whitelisted domain (debian.org).

Proxy and direct-egress block. From a client, browsing through Squid works while a direct connection to the Internet times out, confirming the proxy-only policy:

```
$ curl -x http://100.100.6.3:3128 -I https://debian.org
HTTP/1.1 200 Connection established (Via: proxyserver)
$ curl -I https://debian.org --connect-timeout 5
# DENY / timeout (no direct egress)
```

Reverse proxy by name. After aligning DNS to the reverse proxy, internal clients reach the published services *by name*, transparently through nginx (Figure 3); a GET on the vending machine returns its real page (Figure 4).

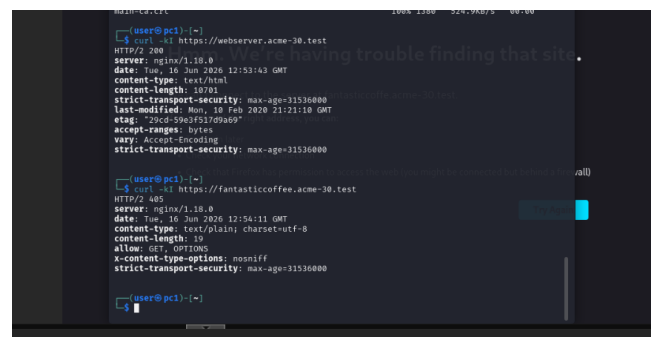


Figure 3. Clients reach the web service and the vending machine by name through the reverse proxy (server: nginx).

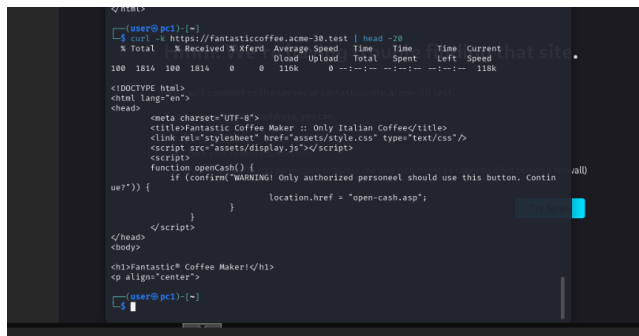


Figure 4. The vending machine page served end-to-end through the reverse proxy over HTTPS.

IPv6 enforcement. Name resolution and access to the reverse proxy work over IPv6 as well (Figure 5), while a direct connection to the backend is blocked, confirming the dual-stack policy:

After the DNS alignment the AAAA record of the name points to the reverse proxy (...1e06:74da...), so the client reaches it *by name*, with no manual override:

```
$ dig -6 webserver.acme-30.test AAAA +short
2001:470:b5b8:1e06:74da:b5ff:fed2:952a # proxy
$ curl -6 -kI https://webserver.acme-30.test/
HTTP/2 200 (server: nginx/1.18.0)
```

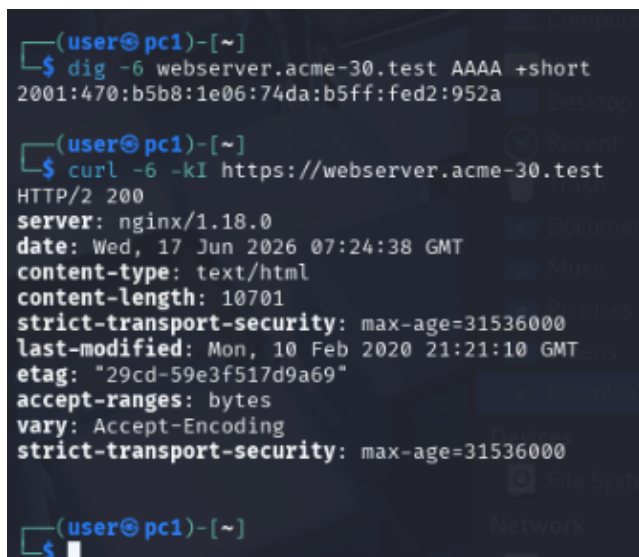


Figure 5. IPv6 end-to-end test from a client: AAAA resolution and access to the reverse proxy over IPv6.

Internet → reverse proxy. From a host on the simulated Internet (no VPN), the published services are reachable *only* through the reverse proxy, while the real backends are not. The proxy presents the internal-CA certificate (TLS terminated there):

```
$ curl -k --resolve webserver.acme-30.test:443:\
100.100.6.3 -I https://webserver.acme-30.test/\
HTTP/2 200 (server: nginx/1.18.0)
$ ... s_client ... | openssl x509 -subject -issuer
subject= CN=webserver.acme-30.test
issuer = CN=internal-ca
$ curl -k --connect-timeout 5 -I https://100.100.6.2/\
curl: (28) Connection timed out # backend: DENY
$ nc -zvn -w3 100.100.1.4 9392
... timed out # Greenbone: DENY
```

The firewall log confirms it: from the same Internet host, the request to the proxy (.6.3:443) passes while the ones to the backend (.6.2:443) are dropped (Figure 6).

Interface	Time	Source	Destination	Proto	Label
wan	→ 2026-06-16T19:20:48	100.101.0.2:39396	100.100.6.2:443	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T19:20:47	100.101.0.2:39396	100.100.6.2:443	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T19:20:46	100.101.0.2:39396	100.100.6.2:443	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T19:20:45	100.101.0.2:39396	100.100.6.2:443	tcp	A3 BLOCK LOG DEFAULT DENY
DMZ	← 2026-06-16T19:20:33	100.101.0.2:48554	100.100.6.3:443	tcp	let out anything from firewall host itself

Figure 6. Internet host: traffic to the reverse proxy (.6.3) is allowed, direct traffic to the backend (.6.2) is dropped.

Default-deny logging. Forbidden traffic (e.g. an Internet host probing the firewall or an internal backend) is dropped and logged by the BLOCK LOG DEFAULT DENY rule, as shown in the firewall live view (Figure 7).

Interface	Time	Source	Destination	Proto	Label
wan	→ 2026-06-16T08:08:16	100.101.0.2:46934	100.100.0.2:80	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T08:08:15	100.101.0.2:46918	100.100.0.2:80	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T08:08:15	100.101.0.2:46906	100.100.0.2:80	tcp	A3 BLOCK LOG DEFAULT DENY
wan	→ 2026-06-16T08:07:43	100.101.0.2:46934	100.100.0.2:80	tcp	A3 BLOCK LOG DEFAULT DENY

OPNsense [c] 2014-2024 Deciso B.V.

Figure 7. Traffic not matching any allow rule is dropped and logged.

Administration. The firewall GUIs, the Greenbone UI (9392) and the Graylog UI are reachable from the admin host (Kali) and denied to ordinary clients. Greenbone can scan the ACME zones, and the *only* connection accepted toward it is the administrator reaching its UI: no other host may initiate any connection to the scanner.

7 Final remarks

The ACME network now enforces a coherent *least-privilege*, *default-deny* policy on both firewalls: every allowed flow maps to one readable, alias-based rule, every other packet is dropped and logged, and the dropped events are centralized in Graylog together with the host and service logs. Public web services are exposed only through a single reverse proxy that upgrades them to HTTPS, internal hosts reach the Internet only through the forward proxy, and the whole policy was replicated on IPv6.

A few hardening steps were already applied beyond the minimum (HTTPS-only web service, syslog over TLS,

ICMPv6 filtering, firewall log forwarding).

References

- [1] K. Scarfone, P. Hoffman, *Guidelines on Firewalls and Firewall Policy*, NIST SP 800-41 Rev. 1, 2009.
- [2] E. Davies, J. Mohacsi, *Recommendations for Filtering ICMPv6 Messages in Firewalls*, RFC 4890, 2007.
- [3] F. Miao, Y. Ma, J. Salowey, *Transport Layer Security (TLS) Transport Mapping for Syslog*, RFC 5425, 2009.

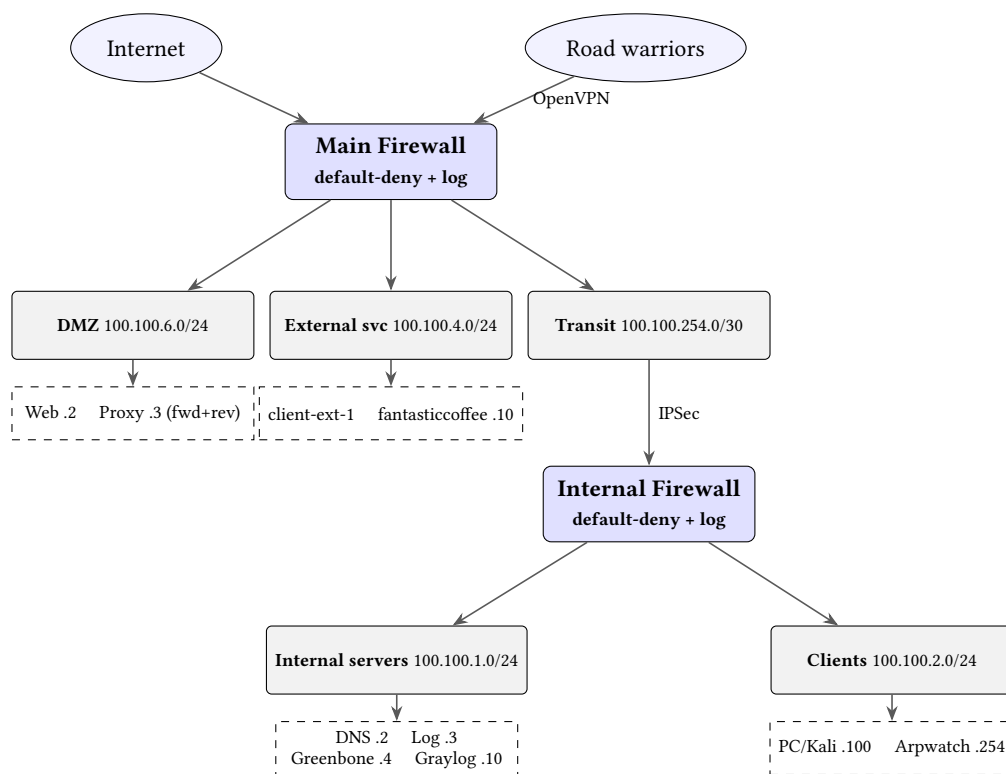


Figure 8. ACME-30 topology. The Main firewall protects the WAN/DMZ/External zones and terminates the road-warrior VPN; the Internal firewall isolates the servers and clients networks. Public web services are exposed only through the reverse proxy in the DMZ.